Supervised Machine Learning: Regression & Classification
Coursera Course 1/3 Notes
Nicolas Rebollo Canedo-Arguelles
Started Dec 14, 2025; Ended Dec 21, 2025

Table of Contents

1. Week 1 Notes - Introduction to ML....3
1.1. What is Machine Learning?....3
1.2. Supervised Learning Part 1....3
1.3. Supervised Learning Part 2....3
1.4. Unsupervised Learning Part 1....4
1.5. Unsupervised Learning Part 2....4
1.6. Jupyter Notebooks....4
1.7. Linear Regression Model Part 1....5
1.8. Linear Regression Model Part 2....5
1.9. Cost Function Formula....6
1.10. Cost Function Intuition....6
1.11. Visualizing the Cost Function....7
1.12. Visualization Examples....7
1.13. Gradient Descent....7
1.14. Implementing Gradient Descent....7
1.15. Gradient Descent Intuition....8
1.16. Learning Rate....8
1.17. Gradient Descent for Linear Regression....8
1.18. Running Gradient Descent....9
1.19. Gradient Descent Implementation in Code (No Vectorization or Feature Scaling)....9
2. Week 2 Notes - Regression with Multiple Input Variables....11
2.1. Multiple Features....11
2.2. Vectorization Part 1....12
2.3. Vectorization Part 2....12
2.4. Gradient Descent for Multiple Linear Regression (with Vectorization)....12
2.5. Feature Scaling Part 1....14
2.6. Feature Scaling Part 2....14
2.7. Checking Gradient Descent for Convergence....15
2.8. Choosing the Learning Rate....15
2.9. Feature Engineering....16
2.10. Polynomial Regression....16

3. Week 3 Notes - Classification.....16
3.1. Motivations.....16
3.2. Logistic Regression.....17
3.3. Decision Boundary.....18
3.4. Cost Function for Logistic Regression.....18
3.5. Simplified Cost Function for Logistic Regression.....19
3.6. Gradient Descent Implementation for Logistic Regression.....19
3.7. The Problem of Overfitting.....20
3.8. Addressing Overfitting.....20
3.9. Cost Function with Regularization.....21
3.10. Regularized Linear Regression.....22
3.11. Regularized Logistic Regression.....22

1. Week 1 Notes - Introduction to ML

1.1. What is Machine Learning?

- *Machine Learning:* Field of study that gives computers the ability to learn without being explicitly programmed (Arthur Samuel, 1959)
- Less games or training leads to worse performance in the model. The more opportunities to learn, the better it will perform.
- 2 main types of machine learning are: supervised and unsupervised learning
- Supervised learning is the one that is used most in real-world application and has seen the most rapid advancements
- Other systems include recommender systems and reinforcement learning

1.2. Supervised Learning Part 1

- Learn X -> Y (input to output) mappings
- Give learning algorithm examples to learn from that include the "right answers" or correct label Y for given input X

Input (x)	Output (y)	Application
email	Spam? (0,1)	Spam filtering
audio	Text transcripts	Speech recognition
english	spanish	Machine translation
Ad, user info	Click? (0,1)	Online advertising
Image, radar info	Position of other cars	Self-driving car
Image of phone	Defect? (0,1)	Visual inspection

- *Regression* (type of supervised learning): predict a number from infinitely many possible outputs

1.3. Supervised Learning Part 2

- Supervised learning algorithms learn to predict input to output mappings
- Regression learns to predict numbers out of infinitely possible numbers
- *Classification Algorithm*: trying to predict only a small number of outputs or categories (usually 2 categories like benign or malignant tumor for breast cancer detection)
- Not continuous prediction like regression
- Can plot classification dataset on a line since only 2 outputs

- Can have more than 2 categories (for example type 1 and type 2 malignant). This would lead to 3 categories on a number line
- In classification, “output classes” and “output categories” are interchangeable
- Classification algorithms predict categories (can be non-numeric, e.g., picture is cat or dog, tumor is benign or malignant).
- Can use more than 1 input value to predict an output (e.g., predict tumor type based on tumor size and age → 2 inputs)
- One way to classify is for the learning algorithm to create a boundary line between the output classes
- RECAP: Supervised learning maps input X to output Y and the learning algorithm learns from being given “right answers”. Two major types of supervised learning are regression (predict numbers from infinitely many output numbers) and classification (prediction of category out of a small set of possible outputs)

1.4. Unsupervised Learning Part 1

- After supervised learning the most widely used form of machine learning is unsupervised learning
- *Unsupervised Learning:* Given data that is not associated with any output label Y (e.g., given data on tumor size and patient age, but no data on whether their tumor was benign or malignant). Unsupervised learning finds something interesting (pattern or structure) in the unlabeled data. Not supervising algorithms to give some right answer for every input.
- Clustering algorithm is a type of unsupervised learning that places the unlabeled data into different clusters or groups and is used in many applications (e.g., used in google news)
- Clustering examples: DNA microarray data to split people into types, grouping customers by why they took a course

1.5. Unsupervised Learning Part 2

- *Unsupervised Learning:* Data only comes with inputs x, but not output labels y. Unsupervised learning algorithms have to find structure in the data
- *Clustering* is a type of unsupervised learning which groups similar data points together
- *Anomaly Detection* is another type of unsupervised learning used to detect unusual events or find unusual data points (fraud detection)
- *Dimensionality Reduction* lets you compress a big dataset into a small one. Compresses data using fewer numbers

1.6. Jupyter Notebooks

- The most widely used tool is Jupyter Notebook. Good for experimenting
- Two types of cells: markdown (text) and code cells

1.7. Linear Regression Model Part 1

- Fitting a straight line to your data
- Any supervised model that predicts a number is a regression model
- Classification model predicts discrete categories (only small number of possible outputs)
- Can use data table to visualize regression data as well (e.g., inputs and outputs as columns)
- *Training Set:* Data used to train the model
- x = "input" variable or feature
- y = "output" variable or "target" variable
- m = total number of training examples
- (x, y) = single training example
- $(x^{(i)}, y^{(i)})$ = ith training example (1st, 2nd, 3rd, mth example)

1.8. Linear Regression Model Part 2

- The training set includes the input features (e.g., size of house) and output targets (e.g., price of house). Output targets are the right answers we will learn from
- Feed training set to learning algorithm, and the algorithm will create some function from the training set, called f (hypothesis, or function)
- f takes a new input x and outputs a new prediction, $\hat{y}$ (estimate/prediction for y)
- f is called the model, x is the input feature, $\hat{y}$ is the prediction
- How to represent f?
- For now, $f_{w,b}(x) = f(x) = wx + b$ (where w and b are numbers which determine the prediction based on the input feature, x
- *w:* weight parameter
- *b:* bias parameter
- Why linear function instead of non-linear? We will use linear for foundation, but can use non-linear later on
- Linear fit is called linear regression with one variable. Where one variable means there is one single input feature, x
- *Univariate Linear Regression:* linear regression with one variable (single input feature, x)

General Notation	Description	Python (if applicable)
a	scalar, non bold	
$\mathbf{a}$	vector, bold	
Regression		
$\mathbf{x}$	Training Example feature values (in this lab - Size (1000 sqft))	`x_train`
$\mathbf{y}$	Training Example targets (in this lab Price (1000s of dollars))	`y_train`
$\mathbf{x}^{(i)}$, $y^{(i)}$	i_{th} Training Example	`x_i`, `y_i`
m	Number of training examples	`m`
w	parameter: weight	`w`
b	parameter: bias	`b`
$f_{w,b}(x^{(i)})$	The result of the model evaluation at $x^{(i)}$ parameterized by w, b: $f_{w,b}(x^{(i)}) = wx^{(i)} + b$	`f_wb`

1.9. Cost Function Formula

- The cost function tells us how well the model is doing so that we may try to improve it
- *w,b:* parameters/coefficients/weights (values you can adjust during training to improve model)
- $\hat{y}^{(i)} = f_{w,b}(x^{(i)}) = wx^{(i)} + b$ is the prediction of the linear regression
- Find w,b such that $\hat{y}^{(i)}$ is close to $y^{(i)}$ for all $(x^{(i)}, y^{(i)})$
- *Cost Function (Squared Error Cost Function):* $J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2$. Extra division by 2 makes future calculation a little bit neater. This is the most commonly used cost function for all regression problems
- Rewrite cost function: $J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$

1.10. Cost Function Intuition

- Model: $f_{w,b} = wx + b$
- Parameters: w, b (different straight lines depending on parameters. Want to find parameters that create a good fit with training data
- Cost function: $J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$
- Goal: $minimize_{w,b}\, J(w,b)$
- J(w) is a function of w (in this simplified example where b=0). Plot J over a range of w, and find the value of w that minimizes J

- How to choose w? Choose smallest J from any given w
- Goal of Linear Regression: minimize $J_{w,b}(w, b)$

1.11. Visualizing the Cost Function

- The plot for cost function with 2 variables (w,b) is a 3D plot. Shape is like a soup bowl. It is a 3D surface plot (varying w and b yields different values for $J_{w,b}$
- Can also plot cost function (3D) with a contour plot
- Center of concentric oval is where J is minimum

1.12. Visualization Examples

- High cost leads to far distance on contour plot from minimum cost

1.13. Gradient Descent

- Have some function $J(w, b)$ (does not have to be a linear regression cost function, it can be any cost function), want min $J(w, b)$
- J(w1,w2,...,wn,b) → minimize J over all of the parameters
- Outline:
    - Start with some guesses for w, b (common choice is w=0, b=0)
    - Keep changing w, b to reduce J(w,b)
    - Until we settle at or near a minimum (note that functions can have > 1 min)
- Can end at different local minima depending on starting position or guess for parameters

1.14. Implementing Gradient Descent

- Algorithm (each step): $w = w - \alpha\frac{\delta}{\delta w}J(w, b)$ AND $b = b - \alpha\frac{\delta}{\delta b}J(w, b)$
- α is the *learning rate* (small positive number between 0 - 1, might be 0.001). Controls how big of a step you take downhill. Large = large steps and vice-versa. Choosing good learning rate is important
- The derivative term tells you in which direction to take the next step and, in combination with the learning rate, it determines the size of the next step
- Repeat this algorithm until convergence (where parameters no longer change much with additional steps)
- *Simultaneously* update w and b (at the same time!)
    - $tmp_w = w - \alpha\frac{\delta}{\delta w}J(w, b)$
    - $tmp_b = b - \alpha\frac{\delta}{\delta b}J(w, b)$
    - $w = tmp_w$
    - $b = tmp_b$

- Important to do it in this order because if you update w directly in the first step, then the w used to calculate b in the second step will be incorrect. You must use the old w to calculate the new b.

1.15. Gradient Descent Intuition

- Simplified example: J(w)
- $w = w - \alpha\frac{\delta}{\delta w}J(w)$
- When derivative is positive, w decreases, and vice-versa

1.16. Learning Rate

- The choice of learning rate has a huge impact on efficiency. Gradient descent may not even work at all
- $w = w - \alpha\frac{\delta}{\delta w}J(w)$
- If α is too small: multiply the derivative term by a really small number (0.000000001). Take very small baby steps. You end up decreasing cost, but incredibly slowly. Very inefficient. So gradient descent WILL work, but may be slow
- If α is too large: multiply the derivative term by a large number. You may skip the minimum point and increase the cost instead of decreasing it. You will likely diverge with huge steps from a large learning rate. So gradient descent may overshoot and never reach the minimum (fail to converge, may even diverge)
- If at a local minima, slope is 0, and gradient descent step will leave the parameter unchanged ($w = w - \alpha \cdot 0 = w$)
- Can reach a local minimum with a fixed learning rate
- Near a local minimum, gradient descent will automatically take smaller steps because the slope (derivative term) will get smaller, leading to update steps being smaller, even if the learning rate, α, is kept at some fixed value

1.17. Gradient Descent for Linear Regression

- Linear regression model: $f_{w,b}(x) = wx + b$
- Cost function: $J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$
- Gradient descent algorithm (repeat until convergence):
  - $w = w - \alpha\frac{\delta}{\delta w}J(w,b)$
  - $b = b - \alpha\frac{\delta}{\delta b}J(w,b)$
- Taking partial derivatives of cost function yields:
  - $\frac{\delta}{\delta w}J(w,b) = \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)}$

- $\frac{\delta}{\delta b}J(w,b) = \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})$

- Gradient descent algorithm for linear regression (repeat until convergence):
  - $w = w - \alpha\frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)}$
  - $b = b - \alpha\frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})$
  - Remember that $f_{w,b}(x^{(i)}) = wx^{(i)} + b$
  - Always update w and b simultaneously
- One issue with gradient descent is it can lead to a local minimum rather than the global minimum (slope becomes 0 at local minimum and no further gradient descent steps update parameters)
- But, for square error cost function, it always has a single global minimum (not multiple local minima). This makes the square error cost function a *convex function*
- *Convex Function:* bowl shaped function that cannot have any local minimum other than the single global minimum
- So long as your learning rate is chosen appropriately, the algorithm will always converge to the local minimum

1.18. Running Gradient Descent

- *"Batch" Gradient Descent:* each step of gradient descent uses all of the training examples instead of just a subset
- Other gradient descent versions that look only at subsets of training data at each update step

1.19. Gradient Descent Implementation in Code (No Vectorization or Feature Scaling)

```
#Function to calculate the cost
def compute_cost(x, y, w, b):

    m = x.shape[0]
    cost = 0

    for i in range(m):
        f_wb = w * x[i] + b
        cost = cost + (f_wb - y[i])**2
    total_cost = 1 / (2 * m) * cost

    return total_cost
```

```
def compute_gradient(x, y, w, b):
    """
    Computes the gradient for linear regression
    Args:
```

```
      x (ndarray (m,)): Data, m examples
      y (ndarray (m,)): target values
      w,b (scalar)    : model parameters
    Returns
      dj_dw (scalar): The gradient of the cost w.r.t. the parameters w
      dj_db (scalar): The gradient of the cost w.r.t. the parameter b
     """

    # Number of training examples
    m = x.shape[0]
    dj_dw = 0
    dj_db = 0

    for i in range(m):
        f_wb = w * x[i] + b
        dj_dw_i = (f_wb - y[i]) * x[i]
        dj_db_i = f_wb - y[i]
        dj_db += dj_db_i
        dj_dw += dj_dw_i
    dj_dw = dj_dw / m
    dj_db = dj_db / m

    return dj_dw, dj_db
```

```
def gradient_descent(x, y, w_in, b_in, alpha, num_iters, cost_function, gradient_function):
    """
    Performs gradient descent to fit w,b. Updates w,b by taking
    num_iters gradient steps with learning rate alpha

    Args:
      x (ndarray (m,))  : Data, m examples
      y (ndarray (m,))  : target values
      w_in,b_in (scalar): initial values of model parameters
      alpha (float):     Learning rate
      num_iters (int):   number of iterations to run gradient descent
      cost_function:     function to call to produce cost
      gradient_function: function to call to produce gradient

    Returns:
      w (scalar): Updated value of parameter after running gradient descent
      b (scalar): Updated value of parameter after running gradient descent
      J_history (List): History of cost values
      p_history (list): History of parameters [w,b]
      """

    # An array to store cost J and w's at each iteration primarily for graphing later
    J_history = []
    p_history = []
    b = b_in
    w = w_in

    for i in range(num_iters):
        # Calculate the gradient and update the parameters using gradient_function
        dj_dw, dj_db = gradient_function(x, y, w , b)

        # Update Parameters using equation (3) above
        b = b - alpha * dj_db
        w = w - alpha * dj_dw
```

```
        # Save cost J at each iteration
        if i<100000:      # prevent resource exhaustion
            J_history.append( cost_function(x, y, w , b))
            p_history.append([w,b])
        # Print cost every at intervals 10 times or as many iterations if < 10
        if i% math.ceil(num_iters/10) == 0:
            print(f"Iteration {i:4}: Cost {J_history[-1]:0.2e} ",
                  f"dj_dw: {dj_dw: 0.3e}, dj_db: {dj_db: 0.3e}  ",
                  f"w: {w: 0.3e}, b:{b: 0.5e}")

    return w, b, J_history, p_history #return w and J,w history for graphing
```

```
# initialize parameters
w_init = 0
b_init = 0
# some gradient descent settings
iterations = 10000
tmp_alpha = 1.0e-2
# run gradient descent
w_final, b_final, J_hist, p_hist = gradient_descent(x_train ,y_train, w_init, b_init, tmp_alpha,
                                                    iterations, compute_cost, compute_gradient)
print(f"(w,b) found by gradient descent: ({w_final:8.4f},{b_final:8.4f})")
```

2. Week 2 Notes - Regression with Multiple Input Variables

2.1. Multiple Features

- One feature linear regression was $f_{w,b}(x) = wx + b$
- Multiple features introduced: x_1, x_2, x_3,...
- $x_j = j^{th}$ feature
- n = number of features
- $\vec{x}^{(i)}$ = features of ith training example (row vector)
- $x_j^{(i)}$ = value of feature j in the ith training example
- $f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + ... + w_nx_x + b$
- E.g., $f_{w,b}(x) = 0.1x_1 + 4x_2 - 2x_4 + 80$
- Notation:
    - $\vec{w} = [w_1\ w_2\ w_3\ ...\ w_n]$ (row vector)
    - b is a number (not vector)
    - $\vec{w}$ *and* b are the parameters of the model

    - $\vec{x} = [x_1\ x_2\ x_3\ \dots\ x_n]$ (row vector)
- $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + b$
- This is called *multiple linear regression* (**not** multivariate regression)

2.2. Vectorization Part 1

- Parameters and features:
    - $\vec{w} = [w_1\ w_2\ w_3]$ (n = 3)
    - b is a number
    - $\vec{x} = [x_1\ x_2\ x_3]$
    - In linear algebra, counting/indexing starts from 1. Python is zero-indexed
- Without vectorization (slow for large n):
    - $f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + b = w[0]x[0] + w[1]x[1] + w[2]x[2] + b$
- With vectorization (using for loop, faster for large n):
    - $f_{\vec{w},b}(\vec{x}) = \left(\sum_{j=1}^{n} w_jx_j\right) + b$
- With vectorization (much faster when n is large):
    - $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b = np.dot(w, x) + b$
    - Numpy dot() function is able to use parallel hardware in computer, making it much more efficient than non-vectorized alternatives

2.3. Vectorization Part 2

- Retrieves w and x from memory and, in a single step, multiplies each w and x pair together in parallel, rather than once each time step. Then the computer takes the 16 products and uses specialized hardware to add them efficiently
- Efficient vectorized algorithms scale well to large datasets

2.4. Gradient Descent for Multiple Linear Regression (with Vectorization)

- $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$
- $J(\vec{w}, b)$ is the cost function
- Repeat until convergence:
    - $w_j = w_j - \alpha\frac{\delta}{\delta w_j}J(\vec{w}, b)$
    - $b = b - \alpha\frac{\delta}{\delta b}J(\vec{w}, b)$
- For one feature (univariate linear regression):

- $w = w - \alpha\frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)}$
- $b = b - \alpha\frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})$
- Remember that $f_{w,b}(x^{(i)}) = wx^{(i)} + b$

- For multiple features (multiple linear regression):
  - $w_1 = w_1 - \alpha\frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_1^{(i)}$
  - ⋮
  - $w_n = w_n - \alpha\frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_n^{(i)}$
  - $b = b - \alpha\frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$
  - Simultaneously update w_j (for $j = 1, \ldots, n$) and b
- *The Normal Equation:* An alternative to gradient descent (to find w,b):
  - Works only for linear regression
  - Solve for w, b without iterations
  - Disadvantages: does not generalize to other learning algorithms, slow when number of features is large (> 10,000)
  - Normal equation method may be used in ML libraries that implement linear regression
  - Gradient descent is the recommended way to solve for w, b

General Notation	Description	Python (if applicable)
a	scalar, non bold	
$\mathbf{a}$	vector, bold	
$\mathbf{A}$	matrix, bold capital	
Regression		
$\mathbf{X}$	training example matrix	`X_train`
$\mathbf{y}$	training example targets	`y_train`
$\mathbf{x}^{(i)}$, $y^{(i)}$	i_{th} Training Example	`X[i]`, `y[i]`
m	number of training examples	`m`
n	number of features in each example	`n`
$\mathbf{w}$	parameter: weight,	`w`
b	parameter: bias	`b`
$f_{\mathbf{w},b}(\mathbf{x}^{(i)})$	The result of the model evaluation at $\mathbf{x}^{(i)}$ parameterized by $\mathbf{w}, b$: $f_{\mathbf{w},b}(\mathbf{x}^{(i)}) = \mathbf{w} \cdot \mathbf{x}^{(i)} + b$	`f_wb`

2.5. Feature Scaling Part 1

- E.g., $\widehat{price} = w_1 x_1 + w_2 x_2 + b$
  - $x_1 = [300, 2000]\ ft^2$ of the home
  - $x_2 = [0,\ 5]$ bedrooms
  - House: $x_1 = 2000,\ x_2 = 5,\ price = \$500k$ (one training example)
  - $w_1 = 50,\ w_2 = 0.1,\ b = 50$
  - $\widehat{price} = 50(2000) + 0.1(5) + 50 = \$100.05M$… Way too much! This is due to the large scale of the parameter w_1 for the x_1 (size) feature
  - $w_1 = 0.1,\ w_2 = 50,\ b = 50$
  - $\widehat{price} = 0.1(2000) + 50(5) + 50 = \$500k$ - more reasonable, matches target
- Therefore, when possible values of a feature are large (size of home), the parameter values will be relatively small, and vice-versa
- To avoid large scaling mismatches (which can lead to bouncing around of gradient descent algorithm), we can scale features to be within the same range

2.6. Feature Scaling Part 2

- How do you actually scale features?
  - One way is to take each value of the feature in each example and divide by the maximum of the range
  - $x_1 = \frac{x_1}{max}$
  - *Mean Normalization:* Find mean of data for each feature. Then, subtract mean from each feature value in each example and divide by the range (max - min):
  - $x_1 = \frac{x_1 - \mu_1}{max - min}$
  - *Z-Score Normalization*: Calculate mean standard deviation of each feature. Then subtract the mean from each feature value in each example and divide by std.
  - $x_1 = \frac{x_1 - \mu_1}{\sigma_1}$
- Rule of thumb: aim for about $-1 \leq x_j \leq 1$ for each feature x_j. -3 to 3, -0.3 to 0.3, or even 0 to 3 are also okay… It is a loose rule. But -100 to 100 or -0.001 to 0.001 need rescaling

2.7. Checking Gradient Descent for Convergence

- Gradient descent:
  - $w_j = w_j - \alpha \frac{\delta}{\delta w_j} J(\vec{w}, b)$
  - $b = b - \alpha \frac{\delta}{\delta b} J(\vec{w}, b)$
- The job is to minimize cost function. Can plot cost function (J) at each gradient descent iteration (x-axis is number of iterations). This is called a *learning curve*
- If gradient descent is working properly, cost function should decrease after every iteration
- If cost function increases after any iteration, either the learning rate (α) is not chosen properly (too large), or could be a bug in the code
- Once learning curve flattens out, the gradient descent algorithm has converged
- Number of iterations varies greatly between applications
- Another way to test convergence of the gradient descent model is with an *automatic convergence test*
- *Automatic convergence test:*
  - *Let* $\varepsilon = 10^{-3} = 0.01$
  - If $J(\vec{w}, b)$ decreases by $\leq \varepsilon$ in one iteration, declare convergence (found parameters $\vec{w}, b$ that are close to the global minimum of $J(\vec{w}, b)$

2.8. Choosing the Learning Rate

- If cost sometimes goes up and sometimes goes down after iterations or always goes up after each iteration, gradient descent not working properly, likely due to a learning rate which is too large → choose a smaller learning rate (and debug code for bugs)
- With a small enough learning rate (α), the cost function, $J(\vec{w}, b)$ should decrease on every iteration
- If, even with learning rate set to a very small number, the cost keeps increasing after iterations, then you likely have code bugs. This is only for debugging as a very small learning rate is not very efficient
- If learning rate (α) is too small, gradient descent takes a lot more iterations to converge
- Values of learning rate (α) to try (in order):
  - 0.001
  - 0.01
  - 0.1
  - 1
  - Plot cost function for each learning rate over a handful of iterations, then pick the learning rate which decrease the cost curve rapidly and consistently (i.e., increase learning rate by 3x until the cost function increases after iterations)
  - A learning rate of 0.1 is a good start for regression with normalized features

2.9. Feature Engineering

- Choice of features is a critical step for making an algorithm work well
- We can use intuition or knowledge about a problem to design new features by transforming or combining original features

2.10. Polynomial Regression

- $f_{\vec{w},b}(x) = w_1 x + w_2 x^2 + b$
- $f_{\vec{w},b}(x) = w_1 x + w_2 x^2 + w_3 x^3 + b$
- $f_{\vec{w},b}(x) = w_1 x + w_2 \sqrt{x} + b$
- Feature scaling becomes increasingly important (e.g., if x is the size of a house and ranges from 1 - 1000 sqft, then x^2 will range from 1 - 1000000, and so on)

> Note the value of **w**, `[0.08 0.54 0.03]` and b is `0.0106`.This implies the model after fitting/training is:
> $$0.08x + 0.54x^2 + 0.03x^3 + 0.0106$$
> Gradient descent has emphasized the data that is the best fit to the x^2 data by increasing the w_1 term relative to the others. If you were to run for a very long time, it would continue to reduce the impact of the other terms.
>
> > Gradient descent is picking the 'correct' features for us by emphasizing its associated parameter
>
> Let's review this idea:
>
> - less weight value implies less important/correct feature, and in extreme, when the weight becomes zero or very close to zero, the associated feature is not useful in fitting the model to the data.
> - above, after fitting, the weight associated with the x^2 feature is much larger than the weights for x or x^3 as it is the most useful in fitting the data.

3. Week 3 Notes - Classification

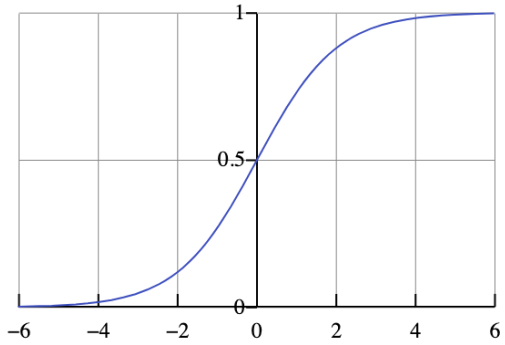
3.1. Motivations

- Linear regression is not a good algorithm for classification algorithms. *Logistic regression* is much better
- Classification examples include: email being spam (no/yes), fraudulent transactions (no/yes), malignant tumor (no/yes)
- *Binary Classification:* Only two possible classes/categories
- Classes usually denoted as no/yes, false/true, 0/1
- The “0” class is the “negative class” and the “1” class is the “positive class”. Positive class conveys presence while negative class conveys absence
- If you want to use linear regression for classification, you can set a threshold value and assume values above the threshold belong to one class and values below belong to another class. However, this is poor when you have outliers as the *decision boundary* shifts over, providing incorrect classification for some values

3.2. Logistic Regression

- Most widely used classification algorithm
- Logistic regression fits an S-shaped curve to classification data on x-y plot
- *Sigmoid Function or Logistic Function:* outputs values between 0 and 1

  $g(z) = \frac{1}{1+e^{-z}}, \ 0 < g(z) < 1$



- Algorithm:
  - Remember from linear regression, $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$
  - Let $z = \vec{w} \cdot \vec{x} + b$ and pass z to the sigmoid/logistic function
  - $f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1+e^{-(\vec{w} \cdot \vec{x}+b)}}$
  - This is the logistic regression model → inputs a feature or set of features, $\vec{x}$, and outputs a number between 0 and 1 from the sigmoid/logistic function
- How to interpret the output of logistic regression:
  - $f_{\vec{w},b}(\vec{x}) = \frac{1}{1+e^{-(\vec{w} \cdot \vec{x}+b)}}$
  - "Probability" that class is 1 given a certain input, $\vec{x}$
  - Example:
    - x is tumor size, y is 0 (not malignant) or 1 (malignant)
    - $f_{\vec{w},b}(\vec{x}) = 0.7 \rightarrow$ 70% chance that y is 1 or tumour is malignant
  - $P(y = 0) + P(y = 1) = 1 \rightarrow$ probability of 0 & 1 always add up to 1 (100%)
  - $f_{\vec{w},b}(\vec{x}) = P(y = 1 | \vec{x}; \vec{w}, b) \rightarrow$ probability that y is 1 given input $\vec{x}$, and parameters $\vec{w}, b$

3.3. Decision Boundary

- $f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = g(z) = \frac{1}{1+e^{-(\vec{w} \cdot \vec{x}+b)}} = P(y = 1|x; \vec{w}, b)$
- How do you predict if the value of f is 0 or 1?
- Can pick a threshold s.t. if $f_{\vec{w},b}(\vec{x}) \geq 0.5$, $\hat{y} = 1$; else, $\hat{y} = 0$
- When is $f_{\vec{w},b}(\vec{x}) = g(z) \geq 0.5$?
    - Occurs when $z \geq 0$ (right side of the y-axis)
    - Which occurs when $\vec{w} \cdot \vec{x} + b \geq 0$
- Linear decision boundary example:
    - Have 2 features x_1 & x_2; $w_1 = 1$, $w_2 = 1$, $b = -3$
    - Decision boundary at $\vec{w} \cdot \vec{x} + b = w_1 x_1 + w_2 x_2 + b = 0$
    - Therefore, $x_1 + x_2 - 3 = 0 \rightarrow x_1 + x_2 = 3$
- Non-linear decision boundary example:
    - $z = w_1 {x_1}^2 + w_2 {x_2}^2 + b$
    - $w_1 = w_2 = 1$, $b = -1$
    - Thus, $z = {x_1}^2 + {x_2}^2 - 1 = 0 \rightarrow {x_1}^2 + {x_2}^2 = 1 \rightarrow$ circle of radius 1

3.4. Cost Function for Logistic Regression

- Squared-error cost function is not ideal for logistic regression
- Squared error cost: $J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \frac{1}{2} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2$
- For logistic regression, using squared error cost makes the cost function non-convex (lots of local minima to get stuck in)
- *Loss of a training example:* $L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)})$
    - For linear regression: $L = \frac{1}{2} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2$
- *Logistic Loss Function:*
    - If $y^{(i)} = 1 \rightarrow L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -log(f_{\vec{w},b}(\vec{x}^{(i)}))$
        - Predicts a loss 0 of as $f_{\vec{w},b} \rightarrow 1$ (prediction same as target)
    - If $y^{(i)} = 0 \rightarrow L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))$
        - Predicts a loss 0 of as $f_{\vec{w},b} \rightarrow 0$ (prediction same as target)

- The further the prediction $f_{\vec{w},b}(\vec{x}^{(i)})$ is from the target $y^{(i)}$, the higher the loss
- This cost function is convex, always leading to a global minimum

The loss function above can be rewritten to be easier to implement.

$$loss(f_{\mathbf{w},b}(\mathbf{x}^{(i)}), y^{(i)}) = (-y^{(i)} \log\left(f_{\mathbf{w},b}\left(\mathbf{x}^{(i)}\right)\right) - \left(1 - y^{(i)}\right) \log\left(1 - f_{\mathbf{w},b}\left(\mathbf{x}^{(i)}\right)\right)$$

This is a rather formidable-looking equation. It is less daunting when you consider $y^{(i)}$ can have only two values, 0 and 1. One can then consider the equation in two pieces:

when $y^{(i)} = 0$, the left-hand term is eliminated:

$$\begin{aligned} loss(f_{\mathbf{w},b}(\mathbf{x}^{(i)}), 0) &= (-(0) \log\left(f_{\mathbf{w},b}\left(\mathbf{x}^{(i)}\right)\right) - (1 - 0) \log\left(1 - f_{\mathbf{w},b}\left(\mathbf{x}^{(i)}\right)\right) \\ &= -\log\left(1 - f_{\mathbf{w},b}\left(\mathbf{x}^{(i)}\right)\right) \end{aligned}$$

and when $y^{(i)} = 1$, the right-hand term is eliminated:

$$\begin{aligned} loss(f_{\mathbf{w},b}(\mathbf{x}^{(i)}), 1) &= (-(1) \log\left(f_{\mathbf{w},b}\left(\mathbf{x}^{(i)}\right)\right) - (1 - 1) \log\left(1 - f_{\mathbf{w},b}\left(\mathbf{x}^{(i)}\right)\right) \\ &= -\log\left(f_{\mathbf{w},b}\left(\mathbf{x}^{(i)}\right)\right) \end{aligned}$$

3.5. Simplified Cost Function for Logistic Regression

- Original logistic loss function (piecewise):
  - If $y^{(i)} = 1 \rightarrow L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -log(f_{\vec{w},b}(\vec{x}^{(i)}))$
  - If $y^{(i)} = 0 \rightarrow L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))$
- Since y can only take on 0 or 1, can rewrite the piecewise as follows:
  - $L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -y^{(i)}log(f_{\vec{w},b}(\vec{x}^{(i)})) - (1 - y^{(i)})log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))$
    - First term cancels out when y is 0, second term cancels out when y is 1
- Simplified logistic cost function becomes:
  - $J(\vec{w}, b) = \frac{1}{m}\sum_{i=1}^{m} L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)})$
  - $J(\vec{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)})log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))\right]$
    - This cost function chosen from maximum likelihood estimation (statistics topic)

3.6. Gradient Descent Implementation for Logistic Regression

- Find $\vec{w}, b$ through gradient descent and then, given new $\vec{x}$, predict $f_{\vec{w},b}(\vec{x}) = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}+b)}}$
- Cost function: $J(\vec{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)})log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))\right]$
- Usual gradient descent algorithm (repeat for each iteration):
  - $w_j = w_j - \alpha\frac{\partial}{\partial w_j}J(\vec{w}, b)$
  - $b = b - \alpha\frac{\partial}{\partial b}J(\vec{w}, b)$

- For logistic regression:
  - $\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$
  - $\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$
  - Use simultaneous updates
  - Therefore
  - $w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right]$
  - $b = b - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \right]$
  - Looks exactly like linear regression gradient descent… Note that this is not linear regression since $f_{\vec{w},b}(\vec{x}^{(i)}) = \frac{1}{1+e^{-(\vec{w} \cdot \vec{x} + b)}}$
- Can monitor gradient descent with a learning curve (cost over iterations)
- Can also vectorize this implementation
- Important to use feature scaling here as well (similar to linear regression)

3.7. The Problem of Overfitting

- *Underfitting data:* data does not fit the training set well (e.g., linear approximation to a clearly non-linear dataset). It can also be said that the algorithm has *high bias*
- *Generalization*: want the learning algorithm to make good predictions (generalize well) for examples not within the training data set. This means the approximation fits training set pretty well (e.g., fit a second order polynomial to a quadratic dataset)
- *Overfitting data:* fits the training set extremely well (e.g., fit a high order polynomial such as 4th order to a quadratic data set; you can hit all examples of your training dataset exactly). However, the curve will be random and wiggly between examples and data points. Therefore, it will not predict house prices outside the training set well. It will not generalize to new examples. It can also be said that the model has *high variance*
- Goal: try to find a model that doesn't have neither high bias nor high variance. Find a model that fits the data just right and generalizes new examples well

3.8. Addressing Overfitting

- One way to address overfitting is to collect more training examples. The fit function will be less variant
- Another way is to use fewer features to reduce overfitting (select the most appropriate set of features through intuition - *feature selection*)
- One disadvantage of feature selection, is that you are throwing away information from unselected features

- Algorithms to automatically choose features exist
- *Regularization*: reduce the size of parameters w_j. Using smaller parameter values or feature coefficients, leads to smaller effect on output and thus less overfitting or variance
- Do not have to regularize b (only w_j). Regularizing b makes small difference
- Summarized, to reduce overfitting, you can:
    - 1. Collect more data
    - 2. Select features (subset)
        - Feature selection in course 2
    - 3. Reduce size of parameters
        - "Regularization"

3.9. Cost Function with Regularization

- If there are smaller values for $w_1, w_2, \cdots, w_n, b$, then that is like having a simpler model that is less likely to overfit (e.g., $w_3 \approx w_4 \approx 0$). Penalize all w_j parameters
- $J(\vec{w}, b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$, where λ is the regularization parameter ($\lambda > 0$)
- By scaling both terms the same way ($\frac{1}{2m}$), choosing a good value for λ is simpler
- By convention, we do not penalize the parameter b, as it makes very little difference on overfitting.
- Some learning algorithm implementations do penalize b by adding $\frac{\lambda}{2m}b^2$ (can exclude)
- Therefore, $min\, J(\vec{w}, b) = min\left[\frac{1}{2m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2\right]$
    - The first term is the mean squared error (linear regression)
    - The second term is the regularization term
- Choosing different values of regularization parameter, λ:
    - $\lambda = 0$, regularization term not used at all, leads to overfitting
    - $\lambda = 10^{10}$, all $w_j \approx 0 \rightarrow f(x) \approx b \rightarrow$ horizontal line → underfits
    - Choose λ that balances fitting data well and keeping w_j small

3.10. Regularized Linear Regression

- $min\, J(\vec{w}, b) = min\left[\frac{1}{2m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2\right]$
- Gradient descent algorithm (repeat every iteration) remains unchanged:
  - $w_j = w_j - \alpha\frac{\partial}{\partial w_j}J(\vec{w}, b)$
  - $b = b - \alpha\frac{\partial}{\partial b}J(\vec{w}, b)$
- The only thing that changes are the partial derivatives, which now become:
  - $\frac{\partial}{\partial w_j}J(\vec{w}, b) = \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)} + \frac{\lambda}{m}w_j$
  - $\frac{\partial}{\partial b}J(\vec{w}, b) = \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \rightarrow don't\ have\ to\ regularize\ b$
- Putting everything together, gradient descent with simultaneous updates becomes:
  - $w_j = w_j - \alpha\left[\frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)} + \frac{\lambda}{m}w_j\right]$
  - $b = b - \alpha\left[\frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})\right]$

3.11. Regularized Logistic Regression

- Looks similar to regularized linear regression
- $J(\vec{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)})log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$
- This penalizes large values of w_j and reduces overfitting
- Gradient descent for logistic regression with simultaneous updates (repeat each iteration):
  - $w_j = w_j - \alpha\left[\frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)} + \frac{\lambda}{m}w_j\right]$
  - $b = b - \alpha\left[\frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})\right]$
    - Looks same as linear regression, except $f_{\vec{w},b}(\vec{x}^{(i)})$ is now for logistic regression with sigmoid function
    - Regularize only w_j and not b (not necessary)